

System Architecture

The system architecture for this deployment begins in GitHub, routes through GCS, and ends in a public API running on GKE. This flow works via code changes that are pushed to the repository, where GitHub Actions handles Continuous Integration (CI) and Continuous Deployment (CD) via GitHub Actions. CI checks that dependencies install, code compiles, and the Docker image can build. CD then authenticates to GCP, builds and pushes the API image to Artifact Registry, and updates the running Kubernetes deployment.

At runtime, the API is served through a Kubernetes LoadBalancer service, which exposes the app publicly on port 80 and routes traffic to the app container on port 8000. The deployment runs two replicas for resilience and uses readiness/liveness probes for health checks. On pod startup, an init container pulls required artifacts from GCS (data files, model file, supporting files) into mounted paths before the FastAPI app starts. This design keeps model and data artifacts separate from code and makes image updates easier, while still letting the service come up with the files it needs.

Key Decisions

I chose a straightforward CI/CD path from GitHub Actions to Google Artifact Registry and then to GKE because the assignment emphasized a working production-style deployment over platform complexity. Keeping the pipeline straight-forward made it easier to debug quickly, prove reproducibility, and show clear evidence of each stage (build, push, deploy, and verify). I also used a dedicated GitHub Actions service account with scoped permissions so the workflow could authenticate non-interactively while still following least-privilege principles.

I intentionally did not use more advanced deployment modes (like blue/green or multi-environment promotion) because they add operational overhead that was not required for this deliverable. Those modes are very useful for reducing release risk at larger-scale, but they require additional infrastructure (such as traffic splitting and multiple services or clusters). For this assignment, a single deployment target with health checks and manual verification gave the best balance of reliability, simplicity, and speed.

I also avoided introducing extra orchestration in CI because the main risk in this project was deployment correctness, not broad feature churn. The better decision here was to make sure the image built consistently, dependencies loaded correctly at runtime, Kubernetes manifests applied cleanly, and required public endpoints responded as expected. In other words, I prioritized a smaller pipeline that was easy to reason about and defend, instead of a more complex one that looked more “enterprise” but was harder to operate within assignment scope.